All Kinds Of Node Depths

AlgoExpert · Very Hard · Binary Trees

The problem

- Sum, over every node, of the depths of all nodes in its subtree (relative to it).
- Per subtree: sumDepths = sumDepths(l) + size(l) + sumDepths(r) + size(r).

Intuition

- Each subtree's depth sum follows from its children's: their sums plus their sizes.

Brute force vs optimal

- Compute depth sum for every subtree separately → $O(n^2)$
- Single post-order returning (size, depthSum, total) → $O(n)$

Approach

- 1. Post-order returns size, depth sum and total
- 2. Depth sum equals children's sums plus their sizes
- 3. Add everyone's depth sums

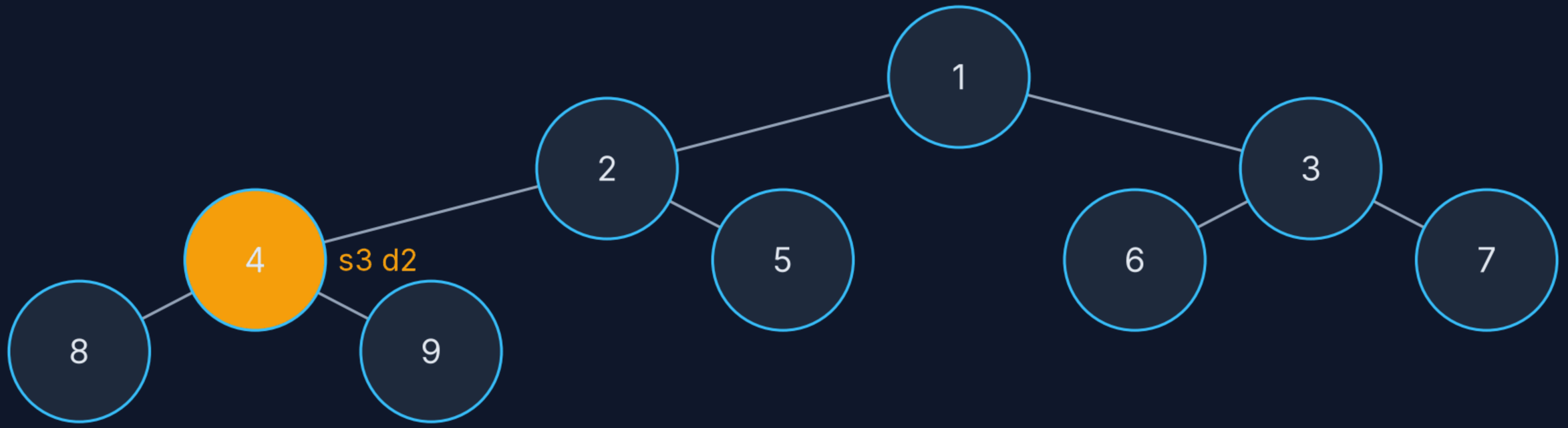
Worked example




```
leaf: size 1, depth-sum 0
```

Leaves: size 1, depth sum 0

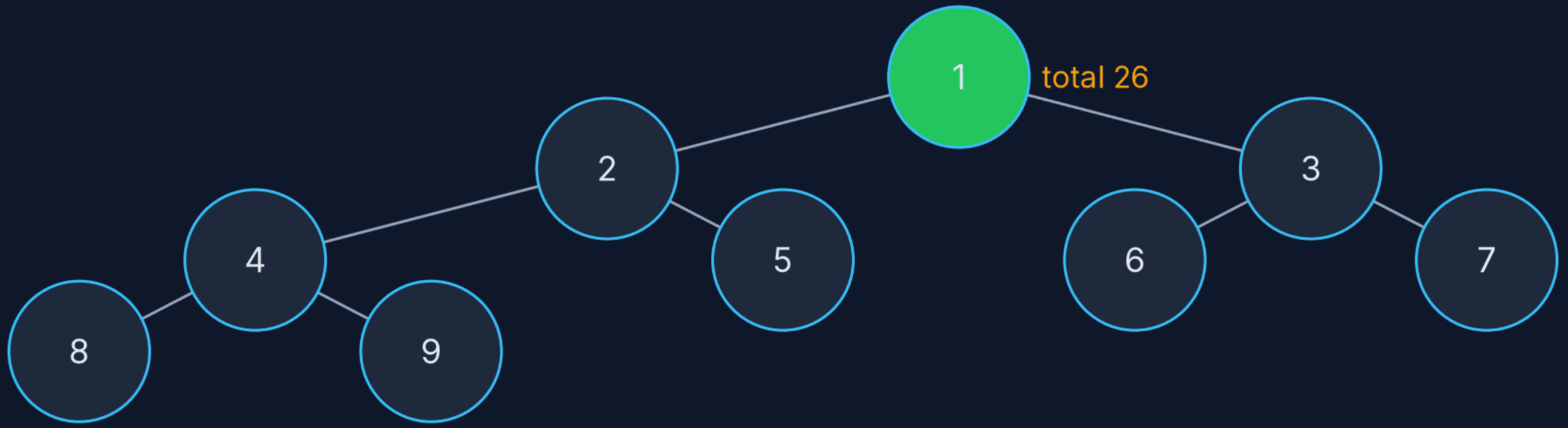
Worked example




```
depths = left depths + left size + right depths + right size
```

Moving up adds 1 per node below

Worked example



`answer = Σ depth sums of all subtrees = 26`

Answer: 26

Java solution

```java
public class AllKindsOfNodeDepths {
    public static int allKindsOfNodeDepths(BinaryTree root) { return (int) info(root)[2]; }

    /** returns {size, sumDepths, total} */
    private static long[] info(BinaryTree t) {
        if (t == null) return new long[]{0, 0, 0};
        long[] l = info(t.left), r = info(t.right);
        long size = 1 + l[0] + r[0], depths = l[1] + l[0] + r[1] + r[0];
        return new long[]{size, depths, depths + l[2] + r[2]};
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Use long for large trees
- Null → zeros

Interview tips & follow-ups

- Explain the +size trick

Recap

- Each subtree's depth sum follows from its children's: their sums plus their sizes.
- Complexity: O(n)